

MINISTRY OF EDUCATION AND TRAINING
HUNG YEN UNIVERSITY OF TECHNOLOGY AND EDUCATION



FINAL PROJECT
DEEP LEARNING
TENNIS COURT KEYPOINTS DETECTION

MAJOR: COMPUTER SCIENCE

STUDENT: VU QUANG PHUC
CLASS: 124221
SUPERVISOR: Associate Prof, PhD NGUYEN MINH TIEN

HUNG YEN – 2025

COMMENTS

Comments from supervisor:This image shows a full page of primary-ruled paper. It features multiple horizontal rows, each defined by two parallel dotted lines. The rows are evenly spaced across the entire page, providing a guide for handwriting practice. There are no margins, text, or other markings present.

SUPERVISOR

(Signature and Full Name)

COMMITMENT

I hereby declare that the course project entitled “Tennis Court Keypoints Detection”, submitted for the Deep Learning Course, is entirely my own work.

All references and materials used in this report have been properly cited in the References section.

All data, figures, and results presented in this project are truthful and accurate.

If any information is found to be incorrect or plagiarized, I will take full responsibility and accept any disciplinary actions imposed by the faculty and the university.

Hung Yen,

Student

ACKNOWLEDGEMENT

To complete this course project, I would first like to express my sincere gratitude to the Department of Software Engineering, Faculty of Information Technology – Hung Yen University of Technical Education, for providing the favorable conditions and support that enabled me to carry out this project successfully.

In particular, I would like to extend my heartfelt thanks to Dr. Nguyen Minh Tien for his dedicated guidance and valuable instructions throughout the process of completing this assignment.

I would also like to express my deep appreciation to all the lecturers of the university who have devoted themselves to teaching and equipping me with the essential and valuable knowledge that made this project possible.

Although I have made great efforts to complete this work, due to my limited knowledge and experience, shortcomings are inevitable. I sincerely hope to receive feedback and constructive comments from my lecturers to help me improve and learn from this experience.

Sincerely,

TABLE OF CONTENTS

COMMITMENT	1
ACKNOWLEDGEMENT.....	2
GLOSSARY OF TERMS.....	4
LIST OF FIGURES.....	5
CHAPTER 1: GENERAL INTRODUCTION.....	9
1.1 Tennis Court Keypoints.....	9
1.2 Planning	10
1.3 My Work.....	11
1.4 Dataset	12
CHAPTER 2: THEORICAL BACKGROUND.....	14
2.1 Deep Learning	14
2.2 Residual Network	15
2.3 OKS (Object Keypoint Similarity)	16
2.4 PCK (Percentage of Correct Keypoints)	18
2.5 OCR (Optical Character Recognition)	18
2.6 Hough Line Transformation	20
CHAPTER 3: IMPLEMENTATION.....	21
3.1 Data collection and labling	21
3.2 Data Splitting (FAISS)	24
3.3 Model Performance	24
3.4 Hyperparameter with Optuna	24
CONCLUSION	27
REFERENCES	28

GLOSSARY OF TERMS

Index	Term	Full form	Meaning
1	CNN	Convolutional Neural Network	
2	DL	Deep Learning	
3	SGD	Stochastic Gradient Descent	
4	RMSE	Root Mean Square Error	
5	PCK	Percentage of Correct Keypoints	
6	ResNet	Residual Network	
7	OKS	Object Keypoint Similarity	

LIST OF FIGURES

Figure 1.1: Data Collection and labeling pipeline.....	12
Figure 1.2: Badminton court (22 keypoints) and tennis court (14 keypoints)	13
Figure 3.1: Directory structure format for Yolo-pose model	21
Figure 3.2: Labeling image data with Roboflow	22
Figure 3.3: Tennis dataset setting at version 1	22
Figure 3.4: Tennis Ground Truth Keypoints Coordinate	23
Figure 3.5: Tennis Predicted Keypoints Coordinate	23
Figure 3.6: Tennis testing results of all CNN architectures	26
Figure 3.7: Visualization on the testing set with ResNet50	26

PREFACE

This project presents the development of an AI application leveraging Computer Vision to automatically summarize tennis and badminton matches from video footage. The long-term goal is to build a complete processing pipeline that includes court detection, player tracking, keypoint and pose estimation, action recognition, and finally, automatic highlight generation.

At the current stage of this report, the team has successfully completed several fundamental milestones: court detection, data annotation, keypoint detection module development (regression-based), and scoreboard content extraction.

The keypoint detection module is formulated as a direct regression problem, predicting the spatial coordinates of keypoints without intermediate heatmaps. Experiments were conducted with multiple backbone architectures, including ResNet, EfficientNet, and several lightweight variants optimized for real-time inference.

The data annotation process follows a custom keypoint format inspired by the COCO keypoint standard, ensuring compatibility and flexibility for future expansion of the training pipeline.

From a technical perspective, the project focuses on the implementation of a regression-based keypoint detection system as a fundamental module within the match summarization pipeline. The main components of the work are summarized as follows:

- **Model Architecture:**

The proposed model adopts a **regression-based keypoint head** built upon a **ResNet50 backbone** pretrained on ImageNet. The final fully connected layer directly predicts the (x, y) coordinates of each court or player keypoint. The primary loss function is the **Smooth L1 Loss**, optionally weighted to emphasize specific keypoints with higher significance. *(Although additional architectures such as EfficientNet or lightweight variants were considered for future experiments, the main implementation and reported results are based on ResNet50.)*

- **Preprocessing and Data Augmentation:**

Input frames are normalized and cropped to focus on the court or player regions. A variety of augmentations are applied—random rotation, flipping, brightness adjustment, and contrast variation—to improve robustness against illumination changes and camera angles.

- **Training Strategy:**

The dataset is divided into training and validation subsets. The training pipeline includes **early stopping**, **model checkpointing**, and hyperparameter tuning for learning rate, weight decay, and batch size to ensure stable convergence. The model is optimized using the **AdamW** optimizer with a cosine annealing learning rate schedule.

- **Evaluation Metrics:**

Two primary metrics are used for evaluating model performance:

- **PCK (Percentage of Correct Keypoints):** Measures the proportion of predicted keypoints that fall within a defined normalized distance threshold from the ground truth. PCK evaluates positional accuracy at specific tolerance levels.
- **OKS (Object Keypoint Similarity):** Computes a similarity score between the predicted and ground truth keypoint sets, normalized by object scale and keypoint variance. OKS provides a holistic measure of keypoint detection quality.

The report details the annotation process, experimental setup (model configuration, learning rate scheduling, and loss functions), and preliminary evaluation results on the validation set using PCK and OKS metrics.

Current challenges include **player occlusion**, **viewpoint and perspective variations**, **low-resolution frames**, and **imbalanced keypoint visibility** due to camera angles or partial body truncation.

In the next phase, the project aims to **integrate a player tracking module** to maintain consistent player IDs across rallies, develop **action recognition and event detection** modules based on frame sequences, and implement **rally segmentation** and **highlight scoring** mechanisms. Further optimization for **real-time inference**—via pruning, quantization, or lightweight network design—is also planned.

The successful completion of the **court detection**, **data labeling**, and **scoreboard extraction** modules establishes a solid foundation for higher-level components within the automatic match summarization pipeline. Future work also includes **advanced data augmentation**, **expanded dataset collection**, and **backbone fine-tuning or ensemble approaches** to improve robustness and increase PCK/OKS performance under real-world conditions.

Keywords: computer vision, keypoint detection, regression-based keypoint, ResNet50, PCK, OKS, court detection, data annotation, tennis, badminton

CHAPTER 1: GENERAL INTRODUCTION

1.1 Tennis Court Keypoints

The main objective of this project is to develop an **intelligent sports video analysis system** capable of automatically identifying and summarizing key moments from full match recordings. The input video is first segmented into **rallies**—continuous exchanges between players—and then analyzed to detect **highlight events** such as **winning points, powerful smashes, or long rallies**.

This is a highly complex computer vision problem that requires the **integration of multiple sub-tasks** to achieve full automation in extracting engaging moments from long sports videos. The goal is to **save viewers' time** and **enhance their watching experience** through AI-driven match summarization.

The main technical modules of the system include:

- **Court Structure Detection:**

Identifies the geometric structure of the tennis or badminton court by detecting key reference points—**14 keypoints for tennis** and **22 for badminton**.

These keypoints are used to normalize the camera view and serve as the foundation for tracking and spatial reasoning.

- **Scoreboard Detection and Recognition:**

Locates the on-screen scoreboard and extracts score information using **Optical Character Recognition (OCR)** techniques, enabling real-time tracking of score changes throughout the match.

- **Object Tracking:**

Tracks the trajectories of players, balls, or shuttlecocks across frames using state-of-the-art tracking algorithms such as **ByteTrack** or **StrongSORT**. This ensures consistent object identity and continuity across time, which is essential for rally and event segmentation.

- **Event Segmentation:**

Segments the match video into individual rallies by detecting the **start and end points** of each exchange—based on serve detection, score change, or other contextual cues.

- **Highlight Generation:**

Combines multiple criteria—such as **rally duration**, **smash speed**, or **critical scoring moments**—to automatically generate **concise highlight videos** summarizing the most exciting parts of the match.

1.2 Planning

The project is divided into four main phases, each consisting of specific sub-tasks with clearly defined objectives and measurable deliverables.

This structured plan ensures both progress consistency and quality assurance, while allowing flexible adjustments based on intermediate outcomes.

Phase 1: Theoretical Research and Survey

- Identify and categorize **common highlight patterns** in tennis and badminton, such as decisive smashes, long rallies with more than 20 exchanges, or spectacular point-winning shots.
- Conduct a **literature review** of related research and modern AI methods, including **YOLO** for object detection, **SORT / ByteTrack** for object tracking, and **Pose Estimation** techniques for analyzing player posture and movement.
- Deliverable: A comprehensive technical survey report summarizing the chosen methods and their applicability to the project.

Phase 2: Data Collection and Annotation

- Collect a dataset of **at least 30–50 tennis matches** and **30–50 badminton matches** in high resolution from open sources such as YouTube or public sports repositories.
- **Annotate objects** including ball/shuttle, players, scoreboard, and court layout using tools like **Roboflow** to ensure high labeling accuracy.
- **Standardize data** following **YOLO** or **COCO** format to facilitate deep learning training.
- Deliverable: A dataset of approximately **1,500 annotated frames per sport**, split into **train/validation/test (70% / 15% / 15%)**, ready for model training.

Phase 3: Object Tracking and Point Detection

- Train **YOLOv8** models integrated with **ByteTrack** or **StrongSORT** to track ball/shuttle and players with high temporal accuracy under dynamic motion conditions.
- Develop and refine **court detection** and **scoreboard recognition** modules, combining OCR (Optical Character Recognition) for automatic score extraction.
- Research techniques for **real-time video segmentation**, using **FFmpeg** for video processing, **GStreamer** for streaming, and **buffer management** for live data handling.
- Deliverable: A tracking system achieving **>10 FPS** and **IoU > 0.5**, along with a **Python script** capable of detecting scoring events and automatically generating highlight clips.

Phase 4: Highlight Generation and Evaluation

- Generate **highlight videos** from segmented rallies based on criteria such as rally duration, smash intensity, and critical scoring moments, utilizing automated video editing tools.
- **Evaluate the highlight quality** through mentor feedback and comparison with manually edited highlight videos from professional sports channels.
- Finalize the **end-to-end system**, fine-tune the trained models based on evaluation results, deliver the complete software package, and compile a **comprehensive final report** documenting the full development cycle.

1.3 My Work

During the internship period, I focused on several key tasks that directly contributed to the overall development of the project. My work was closely aligned with the main phases of the system's implementation, as detailed below:

Phase 1 – Research and Problem Analysis:

Conducted in-depth research and literature review to understand the core challenges of the project. I analyzed relevant studies and existing solutions,

summarized feasible approaches, and proposed suitable methods for integration into the project pipeline.

Phase 2 – Data Collection and Annotation:

Participated in the collection and annotation of diverse, high-quality datasets used for model training. I ensured that the annotated data captured a wide range of match conditions and prioritized high-quality footage from competitive and engaging matches to enhance model generalization.

Phase 3 – Model Implementation and Optimization:

Contributed to the development and training of deep learning models for **court detection** and **scoreboard content extraction**. My focus was on optimizing model performance and integrating these modules into the end-to-end video analysis pipeline to support the overall highlight generation system.

1.4 Dataset

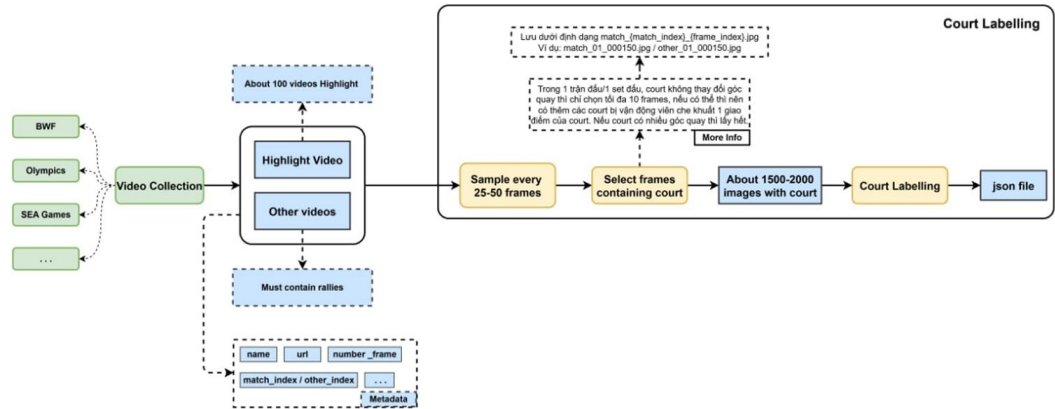


Figure 1.1: Data Collection and labeling pipeline

The dataset consists of **high-resolution images** in either **HD-720p (1280×720)** or **Full HD-1080p (1920×1080)** format. All images were carefully selected from badminton and tennis match videos — including **full matches**, **highlight compilations**, and **summary videos** — ensuring that each frame contains either the full court or partial court areas to maximize diversity and generalization.

Data Sources

The dataset was collected from a wide range of reliable and high-quality sources:

- **Tennis:** Major international tournaments such as **ATP Tour**, **Wimbledon**, and **US Open**, primarily from official YouTube channels or specialized sports video platforms.
- **Badminton:** International tournaments from **BWF (Badminton World Federation)**, **SEA Games**, and **Thomas Cup**, with an emphasis on high-definition videos to minimize visual noise.
- **Highlight Videos:** Additional highlight compilations from sports media such as **ESPN** or **YouTube Sports**, used both for reference in identifying highlight criteria and for expanding dataset diversity.

Dataset Objectives and Composition

- **Tennis:** 1,200 images following similar diversity criteria to ensure robust model training across different environments.
- **Scoreboard:** 2,000 images representing a wide range of visual styles, colors, layouts, tournaments, and player combinations.

Annotation Format

The dataset was annotated using a **custom keypoint-based schema** inspired by the **COCO keypoint format**, tailored for court and scoreboard detection tasks:

1. **Tennis court:** 14 keypoints representing the main court lines and intersections.

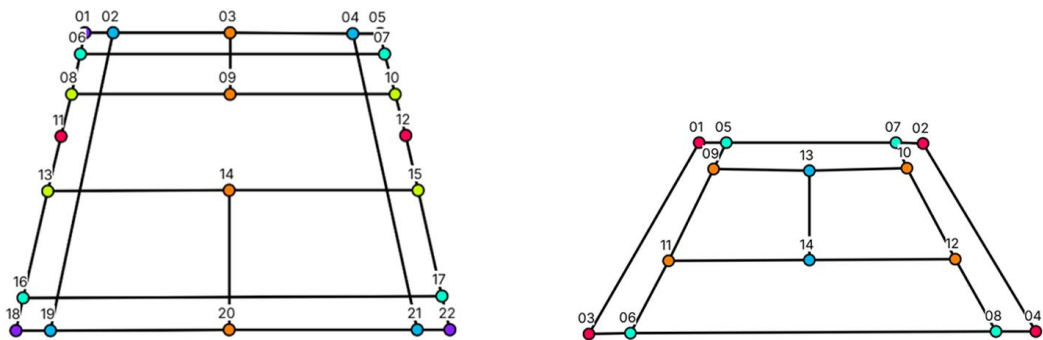


Figure 1.2: Badminton court (22 keypoints) and tennis court (14 keypoints)

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Deep Learning

Deep Learning (DL) is a subfield of Machine Learning (ML) that focuses on computational models composed of multiple layers of nonlinear transformations — commonly known as artificial neural networks (ANNs). It enables computers to learn hierarchical representations of data, allowing complex tasks such as image recognition, speech understanding, and natural language processing to be performed with high accuracy.

At the core of deep learning lies the artificial neuron, inspired by the biological neuron in the human brain. Each neuron receives inputs, multiplies them by weights, adds a bias term, and passes the result through a nonlinear activation function (such as ReLU, sigmoid, or tanh). Layers of neurons are organized sequentially, forming a feed-forward network, where each layer extracts progressively higher-level features from the input data. The earliest layers often capture basic patterns (like edges or textures in images), while deeper layers model abstract concepts (such as object shapes or semantic relationships).

The training process of a deep neural network involves finding optimal weights that minimize a predefined loss function, which measures the difference between the model's predictions and the true targets. This optimization is performed using gradient-based algorithms, primarily Stochastic Gradient Descent (SGD) and its variants such as Adam, RMSProp, and Adagrad. During each iteration, the backpropagation algorithm computes gradients of the loss with respect to all network parameters, enabling efficient weight updates through the chain rule of calculus.

Several important innovations have driven the success of deep learning. These include convolutional neural networks (CNNs), which exploit spatial locality for image analysis; recurrent neural networks (RNNs) and transformers, which capture temporal and contextual dependencies in sequential data; and autoencoders and generative adversarial networks (GANs), which enable unsupervised and generative modeling. Additionally, techniques like batch

normalization, dropout regularization, and residual connections (ResNet) have greatly improved training stability and model performance.

The effectiveness of deep learning also depends heavily on large-scale data and high-performance computing resources, particularly GPUs and TPUs, which allow parallel processing of matrix operations. Frameworks such as TensorFlow, PyTorch, and Keras have made implementation and experimentation more accessible, accelerating both academic research and industrial deployment.

In summary, deep learning represents a paradigm shift from manual feature engineering to automatic feature learning through multi-layered representation.

Its ability to approximate complex nonlinear functions and generalize across diverse domains has made it the foundation of modern artificial intelligence.

Ongoing research continues to focus on model interpretability, efficiency, and generalization, as well as ethical and responsible AI deployment in real-world applications.

2.2 Residual Network

Residual Networks (ResNet) represent one of the most influential architectures in deep learning, addressing the challenges of training very deep neural networks. Proposed by Kaiming He et al. (2015) in the paper “Deep Residual Learning for Image Recognition”, ResNet introduced a simple yet powerful concept — the residual connection — which allows information and gradients to flow more effectively through deep models.

In conventional deep networks, as layers increase, models often face vanishing or exploding gradients, making optimization difficult. Adding more layers sometimes leads to degradation, where training accuracy saturates or even decreases, contrary to the expectation that deeper models should perform better. ResNet overcomes this issue by introducing shortcut (skip) connections that bypass one or more layers, enabling the network to learn residual functions instead of direct mappings.

Mathematically, if the desired mapping is denoted as $H(x)$, a residual block reformulates it as:

$$H(x)=F(x)+x$$

where $F(x)$ represents the residual function learned by a few stacked layers (typically convolution, batch normalization, and ReLU), and $\mathbf{x}$ is the input that is directly added to the output through a skip connection. This formulation allows the layers to focus on learning only the difference between input and output — the residual which is often easier to optimize. The gradient can propagate through the identity path $\mathbf{x}$ without attenuation, alleviating the vanishing gradient problem.

A standard ResNet building block typically consists of two or three convolutional layers with Batch Normalization and ReLU activation, followed by an element-wise addition with the shortcut connection. When the input and output dimensions differ, a 1×1 convolution is used to align their shapes before addition. These residual blocks are stacked to form extremely deep architectures, such as ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152, where the numbers indicate the total number of layers.

ResNet-50, widely used for computer vision tasks, employs a bottleneck design — a three-layer block (1×1 , 3×3 , 1×1 convolutions) that reduces and then restores channel dimensions to improve computational efficiency. This structure balances performance and complexity, making ResNet-50 a common backbone for tasks like image classification, object detection, and keypoint estimation. The success of ResNet has inspired numerous extensions, including ResNeXt, DenseNet, SE-ResNet, and EfficientNet, all building upon the residual learning principle.

Overall, the introduction of residual connections marked a major milestone in deep learning, enabling the training of networks exceeding hundreds or even thousands of layers, and reshaping the design philosophy of modern architectures across vision, language, and multimodal domains.

2.3 OKS (Object Keypoint Similarity)

The **Object Keypoint Similarity (OKS)** metric provides a continuous measure of the similarity between the predicted and ground-truth sets of keypoints.

Unlike PCK, which applies a hard threshold to classify predictions as correct or incorrect, OKS evaluates **the overall alignment quality** by weighting each keypoint according to its labeling difficulty and visibility.

The OKS is defined as:

$$\text{OKS} = \frac{\sum_i \exp\left(-\frac{d_i^2}{2s^2\kappa_i^2}\right) \cdot \delta(v_i > 0)}{\sum_i \delta(v_i > 0)}$$

where:

- d_i : The **Euclidean distance** between the predicted and ground-truth location of the i -th keypoint.
- $s = \sqrt{0.53 \cdot \text{area}}$: The **normalization factor**, with *area* being the bounding-box area of the detected court. This term ensures scale invariance so that large and small courts are evaluated consistently.
- κ_i : The **per-keypoint scale constant**, reflecting annotation uncertainty and the relative importance of each keypoint. For the **tennis court** (14 keypoints), the values used are:
 $[0.075, 0.075, 0.060, 0.060, 0.0725, 0.0725, 0.0575, 0.0575, 0.070, 0.070, 0.055, 0.055, 0.055, 0.040]$
- v_i : The **visibility flag** for the i -th keypoint (1 if the keypoint is visible and annotated, 0 otherwise).
- $\delta(v_i > 0)$: An **indicator function** that ensures only visible keypoints are considered in the summation.

The exponential term penalizes large deviations between the predicted and ground-truth keypoints, with the penalty scaled according to object size and keypoint difficulty.

As a result, OKS values range from 0 to 1, where values closer to **1** indicate a stronger match between prediction and ground truth.

In practice, OKS serves as an analog to **Intersection over Union (IoU)** in object detection, but applied to **keypoint localization**.

It provides a **soft, differentiable measure** that captures both the precision and consistency of keypoint predictions across the entire object.

When used together, **PCK and OKS** offer a comprehensive evaluation framework:

- **PCK** focuses on **threshold-based positional accuracy**, ideal for interpreting performance at specific tolerances.
- **OKS** reflects the **overall geometric and structural consistency**, providing finer insight into how accurately the model predicts the entire keypoint configuration.

2.4 PCK (Percentage of Correct Keypoints)

The **PCK** metric measures the proportion of predicted keypoints that fall within an acceptable distance from their corresponding ground truth locations. It is defined as:

$$\text{PCK}(\alpha) = \frac{\text{Number of Correct Keypoints}}{\text{Total Number of Keypoints}} \times 100\%$$

A keypoint is considered **correct** if the Euclidean distance between the predicted and ground truth locations satisfies:

$$\| p_{pred} - p_{gt} \|_2 \leq \alpha \times L_{norm}$$

where:

- L_{norm} denotes the **normalization length**, defined as the diagonal distance between the **top-left** and **bottom-right** keypoints of the ground truth court.
- α is the **distance threshold coefficient**, typically set to **0.02**, which corresponds to an allowable positional error of approximately **20–30 pixels** in HD-resolution images.

This normalization ensures that the evaluation is **scale-invariant**, allowing fair comparison of keypoint accuracy across different image resolutions and court sizes.

2.5 OCR (Optical Character Recognition)

Optical Character Recognition (OCR) is a fundamental technique in computer vision that enables the automatic extraction and conversion of text from images or video frames into machine-readable form. It is widely used in

tasks such as document digitization, license plate recognition, and visual information retrieval. In the context of this project, OCR plays a crucial role in recognizing scoreboards in sports videos, allowing the system to automatically detect and track score changes over time.

A typical OCR system operates in three main stages. First, preprocessing is applied to enhance image quality and prepare the input for recognition. This includes operations such as grayscale conversion, noise reduction, deskewing, and contrast enhancement techniques like CLAHE. The second stage, text detection, identifies regions containing text using deep learning-based detectors such as EAST, CRAFT, or DBNet, which can handle diverse fonts, orientations, and backgrounds. Finally, in the text recognition stage, the detected text regions are transcribed into digital characters using neural models such as CRNN (Convolutional Recurrent Neural Network) or Transformer-based architectures. Open-source frameworks like EasyOCR, PaddleOCR, or MMOCR are often employed to streamline this process.

In this project, OCR is specifically integrated into the scoreboard detection module. Once the scoreboard is localized, the cropped image is preprocessed and passed through the OCR engine to extract numerical and textual information representing player names and scores. To improve temporal consistency, a post-processing step known as temporal smoothing is applied — for example, using majority voting across consecutive frames to filter out occasional recognition errors and stabilize score updates.

Despite its effectiveness, OCR still faces several challenges in sports video applications. These include low-resolution text, motion blur, complex backgrounds, and non-standard font styles used in broadcast overlays. To address these issues, future improvements may explore end-to-end text spotting models that jointly perform detection and recognition, or employ lightweight OCR architectures optimized for real-time deployment on edge devices and live camera systems.

2.6 Hough Line Transformation

To enhance the accuracy of the predicted keypoints, a post-processing refinement step was applied after the model's initial coordinate prediction. This method utilizes the Hough Line Transform from the OpenCV library to fine-tune the keypoint locations based on the geometric structure of the court.

The refinement process follows several steps. First, for each predicted keypoint, a local square region of 40×40 pixels is cropped around its coordinate (x,y) . This region size is selected to capture sufficient local geometric information, such as the white boundary lines of the court, while minimizing interference from unrelated background regions.

Next, the Hough Line Transform (specifically, the **cv2.HoughLinesP** implementation) is applied to the cropped patch to detect line segments representing the court markings. The Hough Line Transform operates by converting points in image space into a parameterized Hough space, allowing the detection of straight lines by identifying peaks in the accumulator matrix. This process is particularly effective in detecting high-contrast, line-like features, which correspond to the white or bright lines typically found on tennis and badminton courts.

Once the prominent lines are detected, the algorithm computes the intersection point of the two lines that are spatially closest to the original keypoint position. Given two detected lines expressed in slope-intercept form:

$$y = m_1x + c_1 \quad \text{and} \quad y = m_2x + c_2$$

the intersection coordinates $(x_{refined}, y_{refined})$ can be obtained by solving the linear system formed by these two equations. The resulting intersection point replaces the original keypoint coordinate, providing a refined keypoint with improved spatial precision.

CHAPTER 3: IMPLEMENTATION

3.1 Data collection and labling

Video materials were gathered from various open platforms, primarily **YouTube**, focusing on official tournament footage to ensure visual clarity.

For **tennis**, matches from major competitions such as the **ATP Tour**, **Wimbledon**, and **US Open** were selected.

Each video was converted into a sequence of frames using **FFmpeg**, maintaining a frame rate of 2–5 frames per second to balance data diversity and storage size.

Only frames that clearly displayed the court (either fully or partially) were retained. The final dataset consisted of **HD (1280×720)** and **Full HD (1920×1080)** images to ensure sufficient resolution for line detection and keypoint localization.

```
dataset
|__ train
|   |__ images
|   |   |__ image_train_1.jpg
|   |   |__ image_train_2.jpg
|   |__ labels
|   |   |__ image_train_1.txt --> class_id(0) x_min y_min x_max y_max x1 y1
v1(0 or 1 or 2) x2 y2 v2 .....
|   |   |__ image_train_2.txt
|__ val
|   |__ images
|   |__ labels
|__ test
|   |__ images
|   |__ labels
```

Figure 3.1: Directory structure format for Yolo-pose model

These images cover diverse camera angles, court surfaces, and tournament styles, ensuring robust generalization across real-world sports footage.

The final dataset included approximately:

- **1,508 badminton images**, annotated with **22 keypoints** per court instance.
- **1,316 tennis images**, annotated with **14 keypoints** per court instance.
- **2,300 scoreboard images** containing text and numerical data for OCR-based recognition.

Tennis Court Keypoints Detection

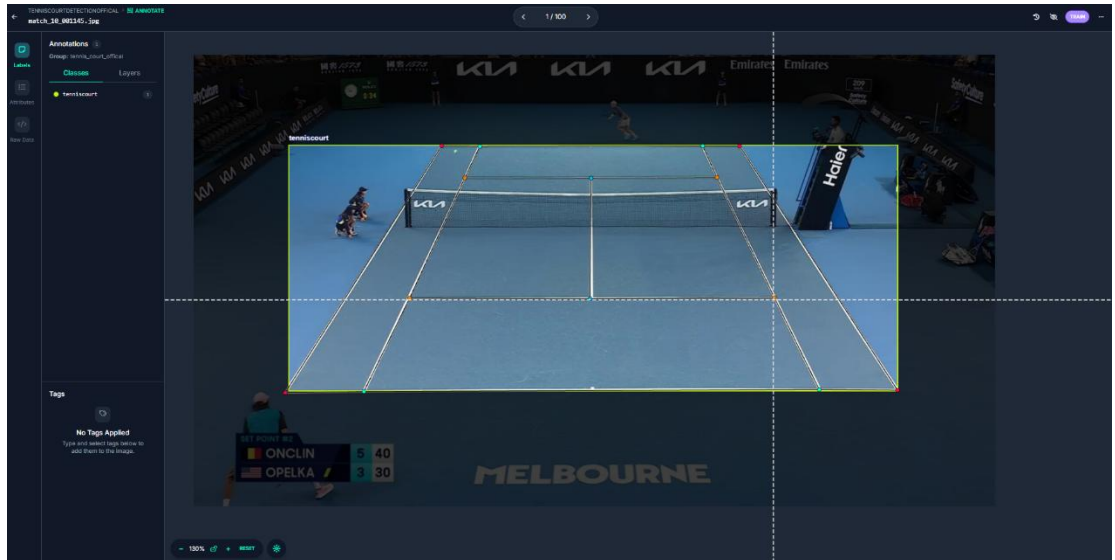


Figure 3.2: Labeling image data with Roboflow

The version displayed (v1) contains 1,351 annotated images used for the tennis court keypoint detection task. The dataset is divided into training, validation, and testing subsets in a 74%–17%–9% ratio, corresponding to 1,000, 233, and 118 images respectively.

Versions + Create New Version

Versions

2025-08-23 2:12am

v1 1351

Phuc Vu Quang (Vũ Quang Phúc 124221)

v1 2025-08-23 2:12am

Generated on Aug 23, 2025 by Phuc Vu Quang (Vũ Quang Phúc 124221)

[Download Dataset](#) [Edit](#)

This version doesn't have a model.

Train an optimized, state of the art model with Roboflow or upload a custom trained model to use features like Label Assist and Model Evaluation and deployment options like our auto-scaling API and edge device support.

[Custom Train](#)

Uses Roboflow Credits [View usage](#)

[How to Upload Custom Weights](#)

1351 Total Images [View All Images](#)

Dataset Split

TRAIN SET	VALID SET	TEST SET
1000 Images	233 Images	118 Images

Preprocessing: No preprocessing steps were applied.

Augmentations: No augmentations were applied.

Figure 3.3: Tennis dataset setting at version 1


```
[
  {
    "id": "test_image_2",
    "bbox": [341, 251, 1247.143, 617.143],
    "area": 769665.386,
    "kps": [
      [ 615, 258 ],
      [ 1312, 258 ],
      [ 345, 860 ],
      [ 1584, 863 ],
      [ 703, 258 ],
      [ 502, 858 ],
      [ 1224, 259 ],
      [ 1425, 861 ],
      [ 675, 345 ],
      [ 1252, 346 ],
      [ 573, 649 ],
      [ 1354, 651 ],
      [ 963, 346 ],
      [ 963, 650 ]
    ]
  },
]
```

Figure 3.4: Tennis Ground Truth Keypoints Coordinate

```
[
  {
    "id": "test_image_2",
    "kps": [
      [608, 259, 1], # GIÁ TRỊ THỨ 3 LÀ CONFIDENCE HOẶC ĐỂ MẶC ĐỊNH LÀ 1
      [1288, 259, 1],
      [331, 851, 1],
      [1566, 851, 1],
      [692, 259, 1],
      [485, 851, 1],
      [1203, 259, 1],
      [1411, 851, 1],
      [663, 342, 1],
      [1232, 342, 1],
      [559, 641, 1],
      [1336, 641, 1],
      [947, 342, 1],
      [947, 641, 1 ]
    ]
  },
]
```

Figure 3.5: Tennis Predicted Keypoints Coordinate

3.2 Data Splitting (FAISS)

To ensure a balanced and representative dataset for model training and evaluation, this project employed **FAISS-based data splitting**, a method designed to maintain diversity and prevent data leakage between training, validation, and testing sets.

FAISS (Facebook AI Similarity Search) is a high-performance library developed by Meta (Facebook AI Research) for efficient **similarity search and clustering** of dense vectors. It allows large-scale feature vectors to be compared and grouped based on their proximity in the feature space. In this project, FAISS was utilized to perform **feature-based stratified splitting**, ensuring that visually similar images (e.g., from the same scene, camera angle, or sequence) are distributed across different subsets without overlap.

3.3 Model Performance

The tennis dataset consists of 1,316 annotated images, each labeled with 14 keypoints representing the structural intersections of the tennis court (such as baselines, service lines, and corners). The dataset follows the same annotation schema as the badminton set, ensuring consistency across experiments.

The model was trained using the following configuration parameters:

- **Batch size:** 16–32, depending on GPU memory availability.
- **Learning rate:** 2×10^{-4} , optimized using the AdamW optimizer.
- **Epochs:** 120 training epochs for each model variant.
- **Data split:** The dataset was divided into training (70%), validation (15%), and testing (15%) subsets to ensure robust evaluation and generalization.

3.4 Hyperparameter with Optuna

The **hyperparameter search space** was defined broadly to cover the most influential factors affecting model performance and generalization. Each parameter range and sampling strategy was carefully selected to balance accuracy, convergence stability, and computational efficiency during training.

- **Learning Rate:**

Defined within the range $[1 \times 10^{-5}, 5 \times 10^{-3}]$ using a **log-uniform distribution**.

This allows exploration of several magnitudes of learning rates while giving more weight to smaller values, which tend to stabilize optimization in deep networks.

- **Batch Size:**

Discrete values $\{16, 32, 64\}$ were considered to balance between **training** throughput and gradient stability. Smaller batches provide noisier but more generalizable updates, whereas larger batches improve GPU efficiency.

- **Image Size:**

Candidate resolutions $\{256, 384, 512, 640\}$ were included to investigate the trade-off between spatial detail and processing time. Higher resolutions can enhance localization precision but increase computational cost.

- **Weight Decay:**

A continuous range $[1 \times 10^{-6}, 1 \times 10^{-3}]$ was used to apply regularization, preventing overfitting by penalizing excessively large weights.

- **Dropout Rate:**

Sampled within $[0.1, 0.4]$ to randomly deactivate neurons during training. This encourages model generalization and reduces dependency on specific activation patterns.

After 250 epochs, the best-performing hyperparameter set was identified, significantly improving performance over the initial baseline configuration.

The optimal parameters are summarized as follows:

- **Batch size:** 32
- **Image size:** 512×512 pixels
- **Learning rate:** 2×10^{-4}
- **Weight decay:** 1×10^{-4}
- **Dropout rate:** 0.25

Tennis Court Keypoints Detection

Model	Epochs	PCK	OKS	RMSE	Inference Time (s)
ResNet18	120	0.837	0.882	11.707	0.0130
ResNet34	120	0.851	0.895	10.758	0.0123
ResNet50	120	0.884	0.907	9.888	0.0111
EfficientNet-B0	120	0.634	0.786	17.566	0.0200
EfficientNet-B1	120	0.705	0.811	15.899	0.0185
MobileNetV2	120	0.795	0.850	13.370	0.0150
MobileNetV3-Small	120	0.680	0.802	16.388	0.0180
MobileNetV3-Large	120	0.706	0.817	15.230	0.0170
VGG16	120	0.822	0.879	11.200	0.0160
VGG19	120	0.888	0.918	8.790	0.0200

Figure 3.6: Tennis testing results of all CNN architectures

This table show all the variants of CNN architectures applied on this project to detect the court keypoints of my team. My work focus on the ResNet50 and apparently it got the result much better than almost other backbone even in transfer learning just below the top first using VGG19 architecture. But my model achieved successfully the **OKS at 0.904**. Really impressive.

This demo following is the visual comparision between my ResNet50 with VGG19 from my team. The result has not made much more difference when come to human eyes.

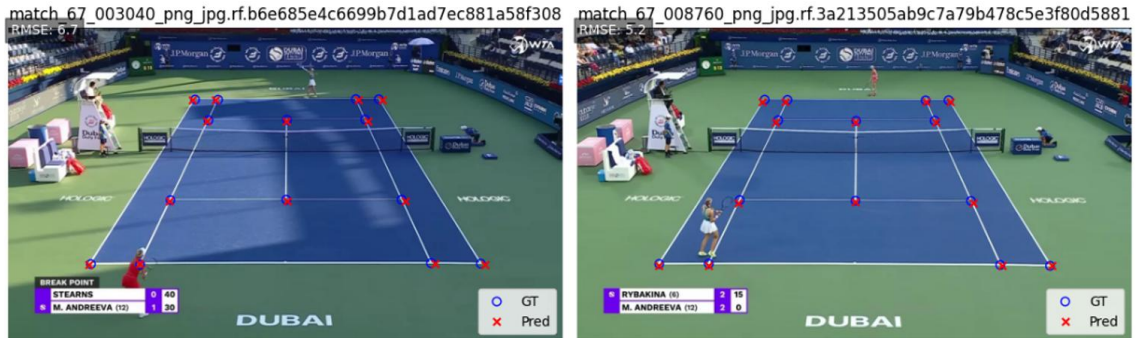


Figure 3.7: Visualization on the testing set with ResNet50

CONCLUSION

This project successfully implemented a deep learning pipeline for detecting tennis court keypoints using ResNet50. The modular structure and configuration-based design enable easy retraining and inference. Although the system performs well, future improvements could include multi-scale feature aggregation, transformer-based backbones, and semi-supervised learning to reduce annotation requirements.

1. Achievements:

- Developed a ResNet50-based regression model for court keypoint detection, achieving high positional accuracy and robust generalization.
- Built and annotated a comprehensive dataset 1,316 tennis images (14 keypoints)
- Integrated an OCR-based scoreboard recognition module, enabling automatic extraction of real-time match scores.
- Implemented a training and evaluation pipeline with early stopping, checkpointing, and performance metrics (PCK and OKS) for systematic model assessment.
- Applied Optuna-driven hyperparameter optimization, improving model convergence and detection accuracy.

2. Future Enhancement:

- Conducted extensive literature research on computer vision techniques for object detection, tracking, and keypoint estimation in sports analysis.
- Designed a modular project pipeline covering data preprocessing, model training, evaluation, and inference for scalable experimentation.
- Established a custom keypoint annotation format inspired by COCO, tailored for multi-sport court geometry.
- Analyzed model performance across diverse conditions (occlusion, lighting, perspective), identifying key limitations and improvement strategies.
- Proposed future enhancements including lightweight model adaptation, temporal modeling, and full end-to-end pipeline automation.

REFERENCES

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. arXiv preprint arXiv:1907.10902, 2019.
- [2] LearnOpenCV. Object keypoint similarity in keypoint detection, 2023.
- [3] Banoth Thulasya Naik, Mohammad Farukh Hashmi, and Neeraj Dhanraj Bokde. A comprehensive review of computer vision in sports: Open issues, future trends and research directions. *Applied Sciences*, 12(9):4429, 2022.
- [4] Roboflow. Roboflow- computer vision data platform, 2020. <https://roboflow.com>.
- [5] Ultralytics. Ultralytics yolo: Real-time object detection and segmentation. 2023. <https://github.com/ultralytics/ultralytics>.
- [6] Yijian Wu, Zewen Chen, Hongxing Zhang, Yulin Yang, and Weichao Yi. Enhanced pose estimation for badminton players via improved yolov8-pose with efficient local attention. *Sensors*, 25(14):4446, 2025.
- [7] Github: https://github.com/lovecrushfamily/Tennis_Court_Keypoints_Detection